

**Brac University**  
**Department of Electrical & Electronic Engineering**  
**Summer 2025**

**Course Number : EEE103L**

Course Title : **Computer Programming Laboratory**

Section       **06**

Group No     : **03**



## Lab Report

---

**Experiment no:**

03

**Name of the experiment:**

Familiarization with Decision Control Structures & Loop Control Structures

*Prepared by:*

Name: Shakila Hoque Nanjiba     ID: 24121125

***All Group members:***

| <b><i>Sl.</i></b> | <b><i>ID</i></b> | <b><i>Name</i></b>             |
|-------------------|------------------|--------------------------------|
| 1.                | 24121283         | Sumaiya Fahmida Zerin          |
| 2.                | 24121346         | Mohammad Sadat Hossain Bhuiyan |
| 3.                | 24121205         | Souvik Barman Ratul            |
| 4.                | 24121125         | Shakila Hoque Nanjiba          |
| 5.                | 24321069         | Tahmid Jamil                   |

### ***Group Contribution:***

All group members equally contributed to coding, testing, logic, and report writing.

### ***Experiment Title and Objective:***

Title:

Problem Solving Using Conditional Statements and Loops in C Programming

Objective:

The objective of this experiment is to develop logical thinking and programming skills by solving real-life problems using C language. The focus is on applying if-else conditions, loops, and user input/output to write programs for grading steel, checking leap years, playing a matchstick game, printing number patterns, and calculating population growth.

### ***Theoretical Background:***

In C programming, we often need to repeat a set of instructions. To do this, we use loops.

#### 1. Loop

A loop allows a block of code to be repeated multiple times until a certain condition is false. This saves time and reduces code repetition.

There are mainly three types of loops in C:

- for loop
- while loop
- do...while loop

#### 2. while Loop

A while loop checks a condition before running the code. If the condition is true, it runs the block. It keeps running until the condition becomes false.

#### 3. if-else Statement

The if-else statement is used for decision making. It checks a condition and runs different code depending on whether the condition is true or false.

#### 4. scanf and printf

- scanf() is used to take input from the user.
- printf() is used to print output on the

screen.

These basic concepts are used in all the programs of this lab to:

- Take input from the user
- Make decisions (like grading or leap year check)
- Repeat actions (like printing patterns or calculating population over years)

***Tasks, Problems, and Solutions:***

a)

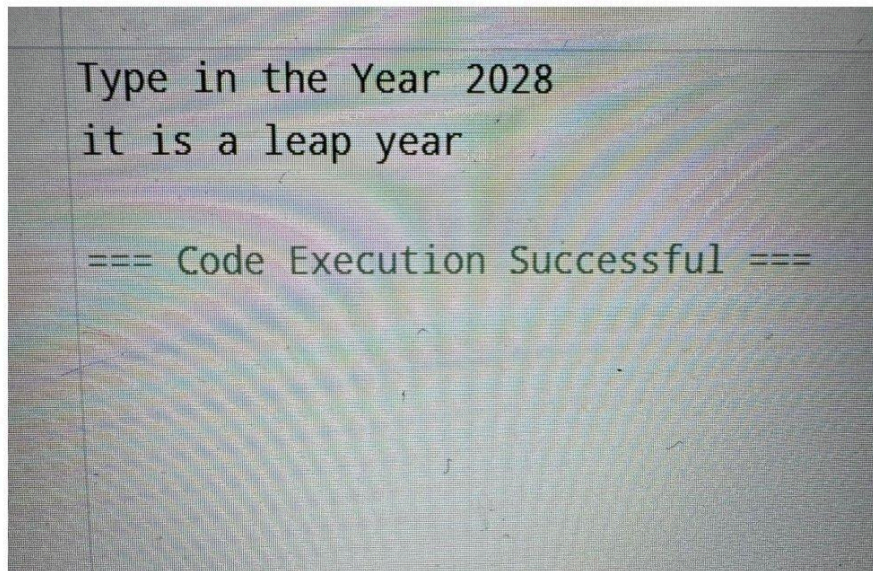
```
#include <stdio.h>

int
main()
{ int
year;
printf("Enter a year: ");
scanf("%d", &year);

int divisibleBy4 = year % 4 == 0;
int notDivisibleBy100 = year % 100 !=
0; int divisibleBy400 = year % 400 ==
0;

if ((divisibleBy4 && notDivisibleBy100) ||
divisibleBy400) { printf("Leap year\n");
} else {
printf("Not a leap year\n");
}

return 0;
}
```



**b)**

```
#include <stdio.h>

int main() {
    double hardness, carbon, strength;

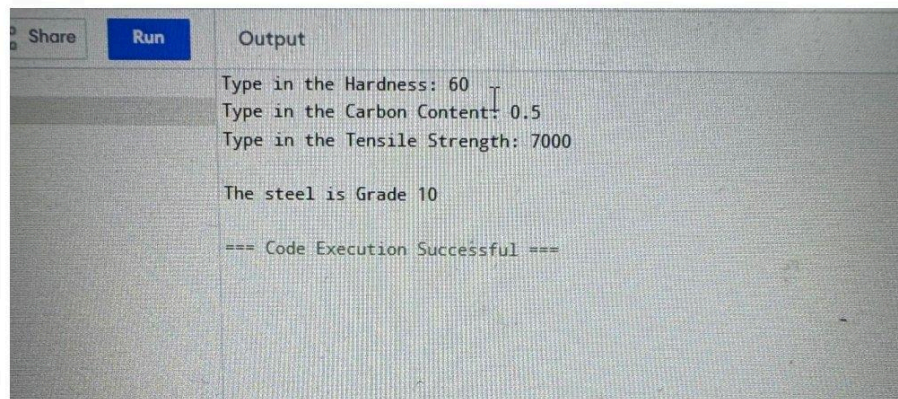
    printf("Enter hardness value:
"); scanf("%lf", &hardness);

    printf("Enter carbon content:
"); scanf("%lf", &carbon);

    printf("Enter tensile strength:
"); scanf("%lf", &strength);

    if (hardness > 50 && carbon < 0.7 && strength >
        5600) { printf("Grade 10 steel\n");
    } else if (hardness > 50 && carbon < 0.7) {
        printf("Grade 9 steel\n");
    } else if (carbon < 0.7 && strength > 5600) {
        printf("Grade 8 steel\n");
    } else if (hardness > 50 && strength > 5600) {
        printf("Grade 7 steel\n");
    } else if (hardness > 50 || carbon < 0.7 || strength > 5600) {
        printf("Grade 6 steel\n");
    } else {
        printf("Grade 5 steel\n");
    }

    return 0;
}
```



The screenshot shows a code execution interface with a 'Share' button, a 'Run' button, and an 'Output' pane. The output pane displays the following text:

```
Type in the Hardness: 60
Type in the Carbon Content: 0.5
Type in the Tensile Strength: 7000

The steel is Grade 10

=== Code Execution Successful ===
```

c)

```
#include <stdio.h>
```

```
int main() {
    int total =
    21; int
    player, bot;

    printf("21 matchsticks are on the table.\n");
    printf("Take 1 to 4 matchsticks on your turn.\n");

    while (total > 1) {
        printf("Your move (1-4): ");
        scanf("%d", &player);

        if (player < 1 || player > 4) {
            printf("Invalid choice. Please choose between 1 and 4.\n");
            continue;
        }

        total -= player;

        bot = 5 - player;
        total -= bot;

        printf("Computer picks %d sticks.\n", bot);
        printf("Remaining matchsticks: %d\n", total);
    }

    printf("Only one stick left. You lost!\n");
}
```



```
    return 0;  
}
```

### Output

```
There are 21 matchsticks.  
Pick 1, 2, 3, or 4 matchsticks.  
Your turn: Pick 1, 2, 3, or 4: 2  
Computer picks 3 matchsticks.  
Matchsticks left: 16  
Your turn: Pick 1, 2, 3, or 4: 4  
Computer picks 1 matchsticks.  
Matchsticks left: 11  
Your turn: Pick 1, 2, 3, or 4: 3  
Computer picks 2 matchsticks.  
Matchsticks left: 6  
Your turn: Pick 1, 2, 3, or 4: 1  
Computer picks 4 matchsticks.  
Matchsticks left: 1  
Uh-oh! Only 1 matchstick left. You lose!
```

```
=== Code Execution Successful ===
```

```
d)
#include <stdio.h>

int main()
{ int n
  = 4; int
  i, j, k;
  int val
  = 1;

  for (i = 1; i <= n; i++) {
    for (j = 1; j <= n - i; j++) {
      printf(" ");
    }

    for (k = 1; k <= i; k++)
      { printf(" %d", val);
        val++;
      }

    printf("\n");
  }

  return 0;
}
```



ever struggle with DSA again.

Learn with **interactive visuals**

Try Now

Output

```
1
2 3
4 5 6
  | 7 8 9 10
```

=== Code Execution Successful ===

```
e)
#include <stdio.h>

int main() {
    int people =
    100000; int
    growth, i;

    printf("Input annual growth rate (in %%): ");
    scanf("%d", &growth);

    for (i = 1; i <= 10; i++) {
        int increase = (people * growth) /
        100; people += increase;
        printf("After year %d: %d\n", i, people);
    }

    return 0;
}
```

SA again. Learn with  
interactive visuals

Try Now

| Run | Output                            |
|-----|-----------------------------------|
|     | Enter growth rate in percent: 10  |
|     | Year 1: 110000                    |
|     | Year 2: 121000                    |
|     | Year 3: 133100                    |
|     | Year 4: 146410                    |
|     | Year 5: 161051                    |
|     | Year 6: 177156                    |
|     | Year 7: 194871                    |
|     | Year 8: 214358                    |
|     | Year 9: 235793                    |
|     | Year 10: 259372                   |
|     | === Code Execution Successful === |

## Discussion

## What We Learned:

In this experiment, we learned how to apply core C programming concepts such as conditional statements (if-else), loops (while, for), and input/output functions (scanf, printf) to solve practical problems. We understood how logic plays a key role in designing programs, such as checking conditions for steel grading, creating patterns, and writing a winning strategy in games. We also became more confident in writing clean and structured code.

## Challenges Faced:

At first, it was difficult to correctly apply multiple if-else conditions in the steel grading program.

- The logic for the matchstick game was tricky because we had to ensure the computer always wins.
- Printing the number pattern with correct alignment and values required careful use of loops.
- Calculating population growth with percentage increase over years needed the correct use of formulas and data types.

## How We Overcame Those Challenges:

We broke down each problem into smaller parts and solved them step by step.

- Discussed and practiced different types of if-else and loop structures to better understand their flow.

Used trial and error by running the code multiple times to fix logic errors.

Took help from class notes, online examples, and discussed doubts with group members.

Overall, this experiment helped us improve our problem-solving skills and gave us hands-on experience with basic programming techniques in C.